Язык программирования C++

Шаблоны

Доцент кафедры ИВТ, к.т.н.
Проскурин Александр Викторович

Обмен местами значений двух переменных

Многие алгоритмы не зависят от типов данных. Например, алгоритм обмена местами значений двух переменных:

```
void swap(double &a, double &b) {
  double t = a;
  a = b;
  b = t;
}

void swap(int &a, int &b) {
  int t = a;
  a = b;
  b = t;
}

void swap(char &a, char &b) {
  char t = a;
  a = b;
  b = t;
}
```

```
int main()
{
  int i = 5, j = 6;
  double d = 8.6, b = 6.7;
  char c1 = 'a', c2 = 'f';
  swap(i, j);
  swap(d, b);
  swap(c1, c2);
  return 0;
}
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Понятие шаблонов

Естественно желание написать алгоритм таким образом, чтобы его можно было использовать для различных типов данных (т.е. выделить **отдельный параметр, отвечающий за тип используемых данных**).

Шаблоны – это средство языка C++, позволяющее давать обобщенные определения **классов** и **функций** без привязки к определенным типам.

С помощью шаблона можно определить алгоритм, который будет применяться к данным различных типов, а **конкретный тип данных передаётся функции в виде параметра** на этапе компиляции. После этого компилятор автоматически **генерирует правильный код**, соответствующий переданному типу.

Механизм шаблонов реализует в C++ **параметрический полиморфизм**.

Шаблоны являются эффективным способом реализации процедур, которые раньше много раз переписывались для данных различного типа.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Определение и вызов шаблона функции

Шаблоны C++ часто называют параметризованными типами. Они позволяют вам компилировать новые классы или функции, задавая типы в качестве параметров.

Функции, созданные по шаблону, называют функциями шаблона или шаблонными функциями.

Обобщённый **синтаксис определения**:

```
template <описание_параметров_шаблона>
тип_возвр_значения имя_функции(параметры_функции)
{
  //Тело функции
}
```

Обобщённый **синтаксис вызова**:

```
имя_функции <аргумент_шаблона>(аргумент_функции);
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Описание параметров шаблона

При объявлении шаблона каждый параметр шаблона может являться:

1. Ключевым словом typename или class, за которым следует идентификатор, обозначающий **параметризированный тип**, например:

```
typename Type        или        class Type
```

В старых версиях typename использовался, если предполагалась передать в качестве параметра встроенный тип данных. Ключевое слово class использовалось в случае передачи пользовательских типов данных.

В новых версиях C++ typename или class являются синонимами.

2. Именем типа, за которым следует идентификатор, например:

```
int a
```

В этом случае в качестве аргумента передаётся конкретное значение, а не тип.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Примеры определений шаблонов функций

- Шаблон функции func воспринимает один параметр произвольного типа:

```
template<typename Type>
void func(Type t)
{ /* Тело функции */ }
```

```
func<int>(5);
func<float>(3.4f);
char ch = 'a';
func<char>(ch);
```

- Шаблон функции func_1 воспринимает два параметра одного произвольного типа:

```
template<typename Type>
void func_1(Type t1, Type t2)
{ /* Тело функции */ }
```

```
func_1<int>(4, 2);
func_1<float>(3.4f, 1.2);
char ch = 'a';
func_1<char>(ch, 'x');
```

- Шаблон функции воспринимает один параметр произвольного типа и одно целое число, которое будет неизменно в течение работы программы:

```
template<typename Type_1, int size>
void func_2(Type_1 t1)
{ /* Тело функции */ }
```

```
func_2<int, 5>(4);
func_2<float, 10>(42.f);
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Автоопределение типа при вызове

При написании программы конкретный тип параметра шаблона задаётся явным образом, либо определяется компилятором автоматически.

В последнем случае нужно быть осторожным и не полагаться на неявное приведение типов.

Например, при таком вызове:

```
func_1(3.4f, 1.2);
```

```
template<typename Type>
void func_1(Type t1, Type t2)
{ /* Тело функции */ }
```

Первый параметр будет определен как тип float, а второй как тип double. Выбор типа шаблона неоднозначен, в связи с чем компилятор выдаст сообщение с ошибкой.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Инстанцирование шаблона

Код шаблона не может быть скомпилирован из-за отсутствия конкретного типа данных.

В связи с этим, компилятор берёт конкретный тип данных, указанный при вызове шаблона, копирует шаблон и подставляет в него выбранный тип.

Таким образом, **создаётся код функции (класса) для соответствующего типа данных**. Чем с большим количеством разных типов вы вызываете шаблон, тем больше будет версий вашего шаблона в итоговом коде, созданном компилятором.

Процесс генерации кода функции или класса по шаблону для конкретных параметров называется **инстанцированием** шаблона (instantiation).

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Объявление шаблона функции

Можно создать прототип шаблона функции в виде его предварительного объявления.

Такое объявление информирует компилятор о наличии шаблона и об ожидаемых параметрах.

Например:

```
template<typename Type_1, typename Type_2>
void func_2(Type_1 t1, Type_2 t2);
```

Однако определение данной функции должно **располагаться в том же файле**. В противном случае компилятор не сможет сопоставить код объявления и определения шаблона функции.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Шаблоны функций. Обмен значений

```
template<typename T>
void swap(T &a, T &b) {
  T t = a;
  a = b;
  b = t;
}
```

```
int main()
{
  int i = 5, j = 6;
  double d = 8.67, b = 6.78;
  char c1 = 'a', c2 = 'f';
  swap<int>(i, j); // Можно указать используемый тип явно
  swap(d, b); // или оставить его определение компилятору,
  swap(c1, c2); // исходя из типов параметров функции
  return 0;
}
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Шаблоны функций. Сортировка

```
template<typename Type>
void sort(Type *array, int size)
{
  for(int i = 0; i < size; i++)
    for(int j = size-1; i > j; j--)
      if(array[j-1] > array[j])
      {
        Type tmp = array[j];
        array[j] = array[j-1];
        array[j-1] = tmp;
      }
}
```

Чтобы применить функцию sort к типу, определенному пользователем (структуре или классу), **требуется перегрузить операции ‘>’ и ‘=’.**

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Шаблоны функций. Перегрузка

```
template<typename Type>
Type getMax(Type t1, Type t2)
{ return t1>t2 ? t1 : t2; }

template<typename Type>
Type getMax(Type *t, int size)
{
  Type res = t[0];
  for(int i = 1; i < size; i++)
    if(t[i] > res)
      res = t[i];
  return res;
}
```

Как и обычные функции, **шаблоны функций могут быть перегружены** как с помощью шаблонов, так и обычными функциями.

```
int main() {
  int i1 = 3, i2 = 5;
  int iarray[] = {3, 9, 5, 8};
  cout << "max int = " << getMax(i1, i2) << endl;
  cout << "max int = " << getMax(iarray, 4) << endl;
  return 0;
}
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Специализация шаблона функции

Можно предусмотреть **специальную обработку отдельных** параметров и **типов** с помощью **специализации шаблона функции**.

К примеру, для С-строк не реализованы перегрузки операций сравнения. Определим шаблон для нахождения максимального значения:

```
template<typename Type>
Type getMax(Type t1, Type t2) { return t1>t2 ? t1 : t2; }
```

И его специализацию, работающую только с С-строками:

```
template<>
char* getMax<char*>(char* t1, char* t2)
{ return (strcmp(t1, t2) == 1) ? t1 : t2; }
```

```
int main() {
  int i1 = 3, i2 = 5;
  cout << "max int = " << getMax(i1, i2) << endl;
  char s1[] = "Golden Eagle";
  char s2[] = "perigrine Falcon";
  cout << "max str = " << getMax(s1, s2) << endl;
  return 0;
}
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Шаблоны классов

Шаблоны создаются для классов, имеющих общую логику работы. Шаблон класса представляет собой скелет обобщенного класса.

Синтаксис объявления шаблонного класса следующий:

```
template<описание_параметров_шаблона>
class имя_класса
{
  // Тело класса
};
```

```
template <typename Type>
class A {
  Type s;
public:
  A(Type _s) : s(_s) {}
  void print(ostream& _out) const;
};
```

В определении класса, входящего в шаблон, имя класса играет особую роль. Оно является не именем отдельного класса, а **параметризированным именем семейства классов**.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Определение метода вне шаблона класса

Методы шаблона класса автоматически становятся шаблонами.

В случае, когда метод шаблона класса описывается вне этого класса (**но обязательно в том же заголовочном файле**), определение метода принимает следующий общий вид:

```
template<описание_параметров_шаблона>
тип_возвр_знач  имя_класса<параметры_шаблона>::
имя_метода(список_параметров_метода)
{
    // Тело
    // метода
};
```

```
template <typename Type>
void A<Type>::print(ostream& _out) const
{
  _out << s << endl;
}
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Объявление объекта шаблонного класса

Синтаксис объявления объекта шаблонного класса имеет следующий вид:

```
имя_класса<аргумент_шаблона> имя_объекта;
```

```
int main()
{
    A<int> iObj(4);
    iObj.print(cout);

    A<float> fObj(1.23f);
    fObj.print(cout);

    A<string> sObj("Hello!");
    sObj.print(cout);

    return 0;
}
```

```
4
1.23
Hello!
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример шаблона класса

```
template<typename Type1, typename Type2>
class myclass
{
  Type1 i; Type2 j;
public:
  myclass(Type1 a, Type2 b) : i(a), j(b) {}

  void show() { cout << i << ' ' << j << '\n'; }
};
```

```
int main()
{
  myclass<int, double> ob1(10, 0.23);
  myclass<char, string> ob2('A', "This is a test");
  ob1.show(); // вывод int, double
  ob2.show(); // вывод char, char *
  return 0;
}
```

Эта программа выдаст следующий результат на экран:
10 0.23
A This is a test

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример шаблона класса

```
class SomeClass // Пример обычного класса
{
  int SomeValue;
  int SomeArray[20];
  ...
};

// Пример шаблона класса
template<int ArrayLength, typename SomeType>
class SomeClassTempl
{
  SomeType SomeValue;
  SomeType SomeArray[ArrayLength];
  ...
};
```

При таком создании количество элементов константно и может быть изменено только перед компиляцией программы.

Аналог класса SomeClass, созданного на основе шаблона:

```
SomeClassTempl<20, int> SomeVariable;
```

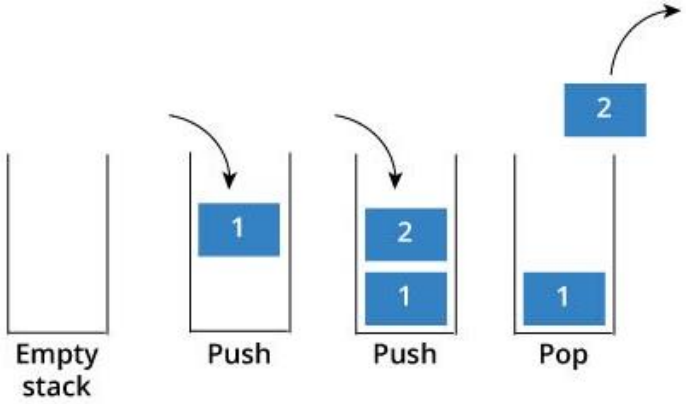
Если нужно 30 элементов массива SomeArray и вещественный тип:

```
SomeClassTempl<30, double> SomeVariable2;
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример шаблона класса Stack (1)

```
template <typename T>
class Stack {
  T* stackPtr; // под элементы
  int size; // размер стека
  int top; // индекс вершины стека
public:
  Stack(int _s = 10) {
    size = _s > 0 ? _s : 10;
    stackPtr = new T[size];
    top = -1; // -1 говорит о том, что стек пуст
  }
  ~Stack() {
    delete[] stackPtr;
  }
  bool push(const T); // поместить элемент в стек
  bool pop(); // удалить из стека элемент
  void printStack(); // вывести элементы на экран
};
```



© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример шаблона класса Stack (2)

```
template <typename T>
bool Stack<T>::push(const T value) {
  if (top == size - 1)
    return false; // стек полон
  top++;
  stackPtr[top] = value; // помещаем элемент в стек
  return true; // успешное выполнение операции
}

template <typename T>
bool Stack<T>::pop() {
  if (top == -1)
    return false; // стек пуст
  top--;
  return true; // успешное выполнение операции
}

template <typename T>
void Stack<T>::printStack() {
  for (int i = top; i >= 0; i--)
    cout << "/ " << stackPtr[i] << endl;
}
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример использования шаблона Stack

```
int main()
{
  Stack<int> myStack(5);

  cout << "Добавляем элементы в стек: ";
  int ct = 0;
  while (ct++ != 5) {
    int temp;
    cin >> temp;
    myStack.push(temp);
  }

  myStack.printStack();

  cout << "\nУдаляем два элемента из стека:\n";
  myStack.pop(); // удаляем элемент из стека
  myStack.pop();

  myStack.printStack();

  return 0;
}
```

```
Добавляем элементы в стек: 5
4
3
2
1
/ 1
/ 2
/ 3
/ 4
/ 5

Удаляем два элемента из стека:
/ 3
/ 4
/ 5
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Правила описания шаблонов классов

- Объявление класса и определение его методов должно находиться в одном заголовочном файле (**нельзя разбивать на *.h и *.cpp файлы**).

- Шаблоны классов могут содержать статические элементы, виртуальные методы, дружественные функции и классы.

- Шаблоны могут быть производными как от шаблонов, так и от обычных классов, а также являться базовыми и для шаблонов, и для обычных классов.

- Можно описывать шаблонные методы в классах, не являющихся шаблонами.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Наследование от шаблона класса (1)

Наследование от шаблона аналогично наследованию от обычного класса. Однако при этом необходимо помнить, что в итоге **вы наследуете от конкретной инстанцированной версии шаблона**. В связи с этим любое упоминание базового шаблона должно сопровождаться перечислением его параметров.

Создадим базовый класс-шаблон:

```
template <typename Type>
class A
{
  Type s;
public:
  A(Type _s) : s(_s) {}
  virtual void print(ostream& _out) const;
};

template <typename Type>
void A<Type>::print(ostream& _out) const
{
  _out << s  << endl;
}
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Наследование от шаблона класса

От него унаследуем другой шаблон:

```
template <typename Type>
class B : public A<Type>
{
  Type m[10];
public:
  B(Type _s, Type _val)
    : A<Type>(_s)
  {
    for (size_t i = 0; i < 10; i++)
      m[i] = _val;
  }

  void print(ostream& _out) const override {
    A<Type>::print(_out);
    for (size_t i = 0; i < 10; i++)
      _out << m[i] << " ";
    _out << endl;
  }
};
```

Обратите внимание на указание типа после названия базового класса. Без него компилятор не сможет создать корректную версию.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример использования шаблона-наследника

```
int main()
{
  cout << "Тип int" << endl;
  B<int> iObj(4, 1);
  iObj.print(cout);

  cout << "Тип float" << endl;
  B<float> fObj(1.23f, 0.0f);
  fObj.print(cout);

  cout << "Тип string" << endl;
  B<string> sObj("Hello!", "!");
  sObj.print(cout);

  return 0;
}
```

```
Тип int
4
1 1 1 1 1 1 1 1 1 1
Тип float
1.23
0 0 0 0 0 0 0 0 0 0
Тип string
Hello!
! ! ! ! ! ! ! ! ! !
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева